

# Notas sobre verificacionismo computacional

Pablo Barenbaum

2026-09-21

§1 El punto de partida es la hipótesis de trabajo, vinculada con la tesis de Church, según la cual la única manera de verificar una afirmación precisa es haciendo cálculos con datos que resulten en una condición de éxito. Podés tomarlo como una definición de lo que para mí significa “verificar una afirmación precisa”.

§2 En una teoría científica (en el sentido positivista/Popperiano), la teoría tiene una forma del estilo “no hay cisnes negros”, y los experimentos tienen el objetivo de falsificar la teoría, o sea de encontrar un cisne blanco. Dualmente, una afirmación precisa tiene una forma del estilo “hay cisnes blancos”, y los experimentos tienen el objetivo de verificarla, o sea de encontrar un cisne blanco.

§3 Definir el sentido de una afirmación precisa consiste en decir qué se necesita para verificarla, o sea, en expresar lo que algunas escuelas del constructivismo llaman *condiciones de verificación*.

§4 Comentario de color: usando terminología lógica, se podría decir algo así como que las teorías científicas están expresadas por fórmulas  $\Pi_1^0$ , que son de la forma  $\forall x.P(x)$ , mientras que las afirmaciones precisas están expresadas por fórmulas  $\Sigma_1^0$ , que son de la forma  $\exists x.P(x)$ , donde acá  $P(x)$  sería un predicado computable. Girard llama “recesivas” a las fórmulas del primer tipo, donde la intuición podría ser que el hipotético contraejemplo que uno busca para refutar la teoría se va alejando a medida que uno hace un experimento que no contradice a la teoría. Y llama “expansivas” a las fórmulas del segundo tipo, en las que el hipotético ejemplo que uno busca para verificar la afirmación se va acercando a medida que uno hace experimentos. Desde ya, se supone que esta discusión acerca de qué significa aseverar una afirmación precisa debería preceder al estudio de la lógica, así que hablar de fórmulas  $\Pi_1^0$  y  $\Sigma_1^0$  es sólo ilustrativo y no fundamental para el desarrollo de este discurso.

Sólo podemos decir que una afirmación precisa tiene sentido si le atribuimos sentido, expresándola como una fórmula expansiva. De modo análogo a como el positivismo o Popperianismo consideraría “pseudocientífica” a una teoría que no sea falsificable, desde esta perspectiva se puede considerar “pseudológica” a una afirmación a la que no seamos cómo atribuirle sentido de tal manera que sea verificable.<sup>1</sup>

§5 La variante computacional del verificacionismo se caracterizaría por que las condiciones de verificación se pueden entender como programas verificadores. Más exactamente, atribuirle sentido a una afirmación precisa  $P$  consiste en asociarle un programa verificador  $\langle P \rangle$ . El programa verificador  $\langle P \rangle$  recibe datos de entrada. Uno puede ponerse a evaluar el programa verificador  $\langle P \rangle$  con ciertos datos de entrada y puede ocurrir que el programa verificador termine exitosamente, lo cual (por definición) da por verificada a la afirmación  $P$ . También puede pasar que el programa verificador dé error o que abandonemos el cómputo antes de llegar a un estado de terminación exitosa, en cuyo caso no logramos verificar  $P$ .

§6 Otro comentario de color: el programa verificador  $\langle P \rangle$  se podría entender como lo que en computabilidad se llama un procedimiento de semi-decisión (que define una función parcial computable). Pero de nuevo, esto es sólo una intuición, porque se supone que esta discusión debería preceder a la teoría de la computabilidad.

§7 Por lo dicho arriba, una afirmación precisa  $P$  es en esencia una forma convencional de abreviar un programa verificador  $\langle P \rangle$ . Aseverar  $P$  equivale a aseverar:

“encontré ciertos datos de entrada  $X$ , evalué  $\langle P \rangle(X)$  y llegué a terminar de manera exitosa”.

---

<sup>1</sup> Aclaro que tengo opiniones encontradas con respecto al positivismo científico, pero me parece que esta puede ser una manera de clarificar qué es o que pare mí constituye una afirmación precisa en el ámbito de la lógica/matemática.

Esta última aseveración encerrada entre comillas ya no es en sí una afirmación precisa, sino una aseveración *primitiva*, que es necesario postular como parte del marco del verificacionismo computacional, junto con un modelo de cómputo conveniente.

§8 Esta definición de qué significa aseverar una afirmación precisa  $P$  en principio se mantiene agnóstica en cuanto a la cuestión del “objetivismo”, es decir, la pregunta de si el resultado que arroja un cómputo está determinado independientemente del observador, el *hardware*, o el contexto en general. Por ejemplo, podría ser que vos logres evaluar  $\langle P \rangle(X)$  exitosamente, y que eso te autorice a aseverar  $P$ , pero que yo no conozca el dato  $X$ , o que no disponga de suficientes recursos computacionales para evaluar  $\langle P \rangle(X)$ , y que por lo tanto no esté en condiciones de aseverar  $P$ .

§9 Esta definición no habla de la posibilidad hipotética de computar  $\langle P \rangle(X)$  y tener éxito, sino que habla de efectivamente haber llevado a cabo ese cómputo. (Podría ser que, adicionalmente al marco verificacionista, alguien esté dispuesto a aceptar una hipótesis de omnisciencia, según la cual el resultado del cómputo podría estar determinado independientemente de que alguien lo lleve a cabo).

§10 Un ejemplo concreto puede ser la afirmación  $P = \text{“existe } n \text{ tal que } n + 1 = 5\text{”}$  a la que podemos atribuirle sentido de manera natural, diciendo que su programa verificador recibe una tira de símbolos, la analiza sintácticamente para chequear que sea una tira de dígitos (fallando en caso de que no lo sea), convirtiendo esos dígitos en un entero que se pueda representar en el modelo de cómputo que se use, haciendo la operación que suma y comparando el resultado con 5, en cuyo caso el verificador tendrá éxito.

Una observación es que mi explicación usa aparentes proposiciones; por ejemplo, “chequear que un dato sea una tira de dígitos” presume aparentemente la proposición “ $X$  es una tira de dígitos”. Pero esto no se trata realmente de una afirmación que requiera a su vez una atribución de sentido. Esto sólo es un abuso de lenguaje para describir una “máquina” (programa verificador). El programa  $\langle P \rangle$  hace la cuenta que determina las condiciones de verificación de  $P$ , y este es el único requisito para atribuirle sentido a  $P$ . Es decir, no es técnicamente necesario explicar qué significa la suma “+” o qué significa la igualdad “=” para darle sentido a  $P$ . Sólo hace falta fijar un modelo de cómputo, y escribir un programa  $\langle P \rangle$  concreto; por ejemplo:

```
def verificador_de_P(evidencia):
    if not evidencia.isnumeric():
        return False
    n = int(evidencia)
    return n + 1 == 5
```

Con esta atribución de sentido, por ejemplo, la afirmación  $P$  se podría verificar exhibiendo el dato de entrada  $X = \text{“4”}$  y haciendo la cuenta  $\langle P \rangle(X)$  que, si todo va bien, debería ser exitosa.

§11 Hay conectivos lógicos problemáticos; en particular la implicación, la negación y la cuantificación universal son problemáticos. Por ejemplo, no queda claro a priori de qué manera se le podría atribuir sentido a la afirmación  $Q = \text{“para todo } n \text{ vale que } n + 1 > n\text{”}$  a través de un programa verificador. El aparente “sentido” intuitivo que le damos a la cuantificación universal requeriría hacer un cómputo haciendo variar  $n$  a lo largo de “todos” los números naturales (con un `while (true) {...}`, digamos). Pero si le proveyéramos una entrada  $X$  a ese programa  $\langle Q \rangle$  ¿nos alcanzarían los recursos computacionales para llegar a un estado de terminación?<sup>2</sup>

§12 Cabe observar también que haberle atribuido sentido a una afirmación precisa  $P$  no significa que quede claro cómo darle sentido a lo que intuitivamente llamamos su negación. Por ejemplo, el sentido de  $P = \text{“existe } n \text{ tal que } n + 1 = 5\text{”}$  se puede expresar por el programa que ya discutimos arriba (es una afirmación expresada por una fórmula expansiva, es decir, podemos tratar de buscar datos para verificarla). En cambio, no es obvio qué sentido le podríamos dar a su negación “no existe  $n$  tal que  $n + 1 = 5$ ” (es una afirmación expresada por una fórmula recesiva, es decir, podemos tratar de buscar datos para refutarla, pero no para verificarla).

---

<sup>2</sup>Notar que desde la perspectiva del sentido común diríamos que el programa  $\langle Q \rangle$  “se cuelga”, mientras que desde la perspectiva del verificacionismo computacional sólo estamos en condiciones de hacer esta pregunta, pero no estamos en condiciones de *negar* que  $\langle Q \rangle$  llegaría a un estado de terminación.

**§13** Por motivos análogos a los del ítem anterior, cuando razonamos acerca de programas informáticos, está claro cómo atribuirle sentido a una afirmación como “el programa  $p$  termina cuando se lo alimenta con la entrada  $x$ ”. El proceso de verificarla consiste en ejecutar el programa  $p$  sobre la entrada  $x$ , resultando en éxito en caso de llegar a un estado de terminación, lo que se puede expresar por una fórmula expansiva. Por otro lado no queda claro de qué manera atribuirle sentido una afirmación como “el programa  $p$  no termina cuando se lo alimenta con la entrada  $x$ ”. El sentido intuitivo que le damos a esta afirmación es recesivo, es decir que podemos buscar una refutación pero no una verificación.

**§14** La vía de escape formalista es la observación de que las afirmaciones se pueden hacer en el contexto de un sistema deductivo formal. En ese contexto, aseverar  $P$  en realidad equivale a aseverar:

“existe un árbol de derivación que concluye  $\vdash P$  en el sistema formal  $X$ ”,

que pasa ahora a ser una afirmación expresable por una fórmula expansiva. El programa verificador de la afirmación  $P$  en realidad resulta ser ahora un *proof-checker* del sistema formal correspondiente. Por ejemplo, a la afirmación problemática de arriba  $Q =$  “para todo  $n$  vale que  $n+1 > n$ ” se le puede atribuir sentido fijando un sistema axiomático específico como la aritmética de Peano o la teoría de conjuntos de Zermelo-Fraenkel y entendiendo que el sentido de  $Q$  es “existe un árbol de derivación que concluye  $\vdash \forall n. n+1 > n$  en PA”. Aseverar  $Q$  ahora es equivalente a aseverar la existencia de una derivación en PA.

**§15** Un problema del formalismo es que las afirmaciones precisas que podemos hacer ya no hablan acerca del procesamiento de los datos que originalmente nos interesaban. Por ejemplo, aseverar  $P =$  “existe  $n$  tal que  $n+1 = 5$ ” ya no quiere decir que conseguí una tira de dígitos que metí en la calculadora para hacer la cuenta  $n+1$  y obtuve 5 como resultado. Significa otra cosa diferente; habla acerca de datos de otra naturaleza que son esos árboles de derivación.

**§16** Otra característica que puede ser vista como problemática del formalismo es que el sentido de las proposiciones no resulta ser composicional. Desde el desiderátum de la composicionalidad, cada conectivo lógico se debería poder entender de manera “funcional”, como la transformación que asocia los sentidos de las subfórmulas inmediatas al sentido de la fórmula entera. Por ejemplo, el sentido de una conjunción  $A \wedge B$  se debería poder entender como dado en función de los sentidos de  $A$  y de  $B$ . Pero esto no es posible a priori desde la perspectiva formalista; los sistemas deductivos formales nos obligan a atribuirle sentido, de alguna manera, a “todas las proposiciones al mismo tiempo”, a partir de la definición del programa que verifica si una derivación está bien construida.

**§17** Un tercer problema, relacionado con el anterior, es que el sentido de un conectivo cambia por completo cuando el sistema se extiende con nuevos conectivos.

**§18** El problema concreto que se presenta es el de encontrar otras posibles maneras de darle sentido a los conectivos lógicos que sean compatibles con esta perspectiva de verificacionismo computacional.

**§19** A modo de nota más personal: estuve trabajando en una manera de abordar el problema de (18) que se podría decir que me satisface<sup>3</sup> y que logra abarcar los conectivos proposicionales y la lógica de orden superior. Pero un problema que tengo al tratar de extender esto a sistemas con inducción como la aritmética de Peano es que el principio de inducción funciona como una regla de introducción del cuantificador universal, que *debilita* el sentido de  $\forall n. P(n)$ , de tal manera que la regla de eliminación usual no está en armonía con las reglas de introducción (como en el caso del operador *tonk*). Esto es lo que me lleva a pensar en una variante de la aritmética con dos cuantificadores universales: uno que no se puede demostrar por inducción y tiene la regla de eliminación usual, y otro que sí se puede demostrar por inducción pero tiene una regla de eliminación más estricta.

**§20** La búsqueda de “consistencia” es una búsqueda estéril, y creo que el problema va más allá del segundo teorema de incompletitud de Gödel. La consistencia de un sistema deductivo formal  $X$  se define en general como la imposibilidad de derivar un seciente como  $\vdash \perp$  en el sistema  $X$ . Pero esta es una afirmación recesiva: dice algo como que “no hay ningún cisne negro”, y por lo tanto es una afirmación pseudológica. Sí, en cambio, tiene sentido la pregunta de si un sistema  $X$  es inconsistente, es decir, la pregunta de hallar una derivación de  $\vdash \perp$  en el sistema  $X$ . (También, desde luego, tiene sentido la pregunta de hallar una derivación de  $\vdash \text{Con}(X)$  en el sistema  $Y$ , donde  $\text{Con}(X)$  es una fórmula que codifica la consistencia de  $X$ ; pero eso a priori habla de la existencia de un árbol de derivación en otro sistema diferente).

---

<sup>3</sup><https://arxiv.org/abs/2605.16157>